

COS 135 Individual Project 1 – Terminal Battleship

Due: Monday 04/06/26 End of the day

What to submit:

- **.c source code.**
- **A two minutes video demonstration of your code, algorithms, and game play.**
- Your programs must produce similar outputs, as given, if the same inputs are provided.
- Program must compile without errors or warnings (use -Wall switch in gcc compiler).

Comments are required in the following locations (in each C source code):

- At the top of the source code comment your name and a short program description.
- Comment the purpose of each variable.
- Comment major sections of code such as functions, inputs, algorithms, and outputs.

Program Design: Your program is a professional document and must be neat and easy to read. All programs should follow these specifications.

- Comments should be aligned and entered in a consistent fashion
- Blank lines should be added to aid readability
- Code within blocks should be indented
- Comments should not contain spelling mistakes
- Variable names, function names, and other identifiers should be meaningful
- Use loops, arrays, and functions appropriately
- Handle invalid user input gracefully

Write a C program for the following project and submit your source codes (you must submit a .c file **without compilation warnings or errors**). Sample Program inputs/outputs are given below.

Use the -Wall switch in gcc to enable all warnings:

```
$ gcc battleship.c -o battleship -Wall
```

Important:

- You are **NOT ALLOWED** to use C concepts that we have not covered in the course.
- Your code **MUST be based on the provided skeleton code**. You are only allowed to complete and modify the given structure (do not rewrite the entire program).
- **Do NOT use** pointers, structs, dynamic memory, or external libraries other than the ones given in the skeleton code.
- Programs that use advanced concepts not covered in class **may receive a grade reduction**.

Project Description

You are tasked to design and develop a Terminal Battleship Game. This is a single-player game played entirely in the Ubuntu terminal. The program should simulate a 6x6 game board and randomly place 5 ships on the board. The player will guess the ship locations by entering row and column numbers. The game also includes a bombing feature that allows the player to reveal multiple cells at once.

Your Terminal Battleship game should have the following features:

(a) Board Setup [20 pts]

- Board size: 6x6
- Number of ships: 5
- Randomly placed ships on the board (using a 2D array)
- No overlapping ships (each ship has its own coordinates)

(b) Game Play [20 pts]

- The player guesses locations by entering row and column numbers (regular guess or a bomb)
- Indicate if a guess is a hit or miss
- Display the board showing only hits (X) and misses (0) – uppercase X and number 0
- Game ends when all ships are completely sunk

(c) Bomb Feature [20 pts]

- The program should allow the player to choose between a regular guess (g followed by row and column - e.g., g 2 3) or using a bomb (b followed by row and column - e.g., b 1 4)
- Player has 2 bombs for the entire game
- A bomb hits a 3x3 area centered on the chosen cell
- Once a bomb hits, reveal the 3x3 area using hits (X) or misses (0)
- Update the remaining bomb count after each use
- Do not allow bombing outside the board (keep track of boundaries)
- For bombs near the edges (0 or 5 row/column), the program should not access invalid array indices.

Example board display:

	0	1	2	3	4	5
0	~	~	~	~	~	~
1	~	~	~	~	~	~
2	~	~	~	~	~	~
3	~	~	~	~	~	~
4	~	~	~	~	~	~
5	~	~	~	~	~	~

Symbols:

- ~ hidden cell
- X hit
- 0 miss

Task Requirements

1. Initialize the Game Board
 - a. Represent the board as a 6x6 grid.
 - b. Display ~ for hidden cells (all hidden in the beginning), X for hits, and 0 for misses.
2. Place Ships Randomly
 - a. The program should place 5 ships randomly on the grid.
 - b. Ships placement should be different each time the program runs.
 - c. Use srand() to ensure different ship placements across runs
 - d. Ensure ships do not overlap (not on same coordinates).
3. User Input and Validation
 - a. Prompt the player to enter row and column numbers ('g' or 'b' with 0 - 5).
 - b. Validate that the input is a number within range.
 - c. If the user enters invalid input, display an error and ask again.
4. Game Logic
 - a. If the user hits a ship, display Hit! and mark X on the board.
 - b. If the user misses, display Miss! and mark 0.
 - c. Prevent shooting the same cell twice; display a warning if attempted.
 - d. A bomb affects 3x3 area centered at the chosen coordinate (row-1 to row+1 and column-1 to column+1). The program must ensure it does not access locations outside the board.
 - e. Each guess counts as one attempt and each bomb use counts as one attempt
5. End of Game
 - a. The game ends when all 5 ships are sunk.
 - b. At the end, display the number of attempts made by the player. Consider a bomb as one attempt.
6. Video Demonstration (~2 minutes):
 - a. **Clearly explain your code in your own words.** Your explanation should include:
 - i. An overview of how your program/code works
 - ii. The main algorithms used (e.g., ship placement, hit/miss logic, bomb logic)
 - iii. The purpose of your key functions and how they interact
 - iv. How you handled input validation and edge cases
 - b. Show your program compiling using -Wall
 - c. Demonstrate live gameplay (at least a few moves, including a hit/miss or bomb)

Note: You should be able to explain your code clearly without reading from a script. Videos that show code without explanation or demonstrate lack of understanding may receive reduced credit.

Video Recording Instructions (2 minutes)

You can record your video demonstration using 1) Zoom, 2) in-built screen recording tools, or 3) a phone camera. The video must clearly show your code running in the Ubuntu terminal while you explain.

Note: make sure audio is recording properly, especially if you are using the option 2.

Option 1: Zoom (desktop sharing while recording)

- Share your screen and record while demonstrating your terminal session.

Option 2: Built-in Screen Recording Tools

Windows 10/11

Approach 1: Snipping Tool

- Search for Snipping Tool in the Start menu and open it.
- Click the Video Camera icon (Record) in the toolbar.
- Click New and drag to select the screen area to record.
- Adjust microphone and system audio settings, then click Start.
- Click Stop (red square) to end, then save the recording.

Approach 2: Xbox Game Bar

- Press Win + G to open the Xbox Game Bar.
- Click the Record button (or press Win + Alt + R) to start recording.
- Record your terminal session and explanation.
- Press the same keys to stop; the video will be saved in Videos > Captures.

Mac (macOS)

- Press Shift + Command + 5 to open the screen recording toolbar.
- Select Record Entire Screen or Record Selected Portion.
- Click Record, demonstrate your code, and explain.
- Press Stop in the menu bar; the video saves to your desktop by default.

Ubuntu

- Press Ctrl + Alt + Shift + R to start/stop recording (works in Ubuntu with GNOME desktop).
- Video is saved in the Videos folder.

Option 3: Phone Camera

- Place your phone so it clearly captures your computer screen while recording your voice.
- Make sure the screen is readable and text is visible.
- Keep video steady and clear.

Sample program execution:

```
cos135@NRC: /media/sf_COS135_VM_Codes/P$ ls
battleship.c
cos135@NRC: /media/sf_COS135_VM_Codes/P$ gcc battleship.c -o battleship -Wall
cos135@NRC: /media/sf_COS135_VM_Codes/P$ ./battleship
Welcome to Terminal Battleship!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): g 1 2
Hit!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): g 3 5
Miss!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ 0
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit):
```

```
Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): g 3 5
Miss!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ 0
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): b 4 1
Bomb used at (4,1)!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ ~
2 ~ ~ ~ ~ ~
3 0 0 0 ~ ~ 0
4 0 0 0 ~ ~ ~
5 0 0 X ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 1 left), or q (to quit): b 4 4
Bomb used at (4,4)!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ ~
2 ~ ~ ~ ~ ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 1 4
Hit!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ ~ ~
3 0 0 0 X 0 0
```

```
cos135@NRC: /media/sf_COS135_VM_Codes/P
0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ ~ ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 2 3
Miss!

0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ 0 ~ ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 2 4
Miss!

0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 0 0
Miss!

0 1 2 3 4 5
0 0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit):
```

```
cos135@NRC: /media/sf_COS135_VM_Codes/P
1 ~ ~ X ~ X ~
2 ~ ~ ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 0 0
Miss!

0 1 2 3 4 5
0 0 ~ ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ ~ ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 0 1
Miss!

0 1 2 3 4 5
0 0 0 ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ X ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 2 1
Hit!

0 1 2 3 4 5
0 0 0 ~ ~ ~ ~
1 ~ ~ X ~ X ~
2 ~ X ~ 0 0 ~
3 0 0 0 X 0 0
4 0 0 0 0 0 0
5 0 0 X 0 0 0

Congratulations! All ships sunk in 10 attempts.
cos135@NRC: /media/sf_COS135_VM_Codes/PS$
cos135@NRC: /media/sf_COS135_VM_Codes/PS$
```

Errors or Invalid inputs:

```
cos135@NRC: /media/sf_COS135_VM_Codes/P
Welcome to Terminal Battleship!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): n 2 3
Invalid action. Enter 'g', 'b', or 'q'.

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): g 7 8
Invalid input.
Miss!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ ~

Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): b 5 5
Bomb used at (5,5)!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ 0 0
```

```
cos135@NRC: /media/sf_COS135_VM_Codes/P
Enter: g row col (to guess), b row col (to bomb, 2 left), or q (to quit): b 5 5
Bomb used at (5,5)!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ 0 0
5 ~ ~ ~ ~ 0 0

Enter: g row col (to guess), b row col (to bomb, 1 left), or q (to quit): b 6 9
Invalid input.
Bomb used at (6,9)!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ 0 0
5 ~ ~ ~ ~ 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): g 4 5
You already guessed that cell.
Miss!

  0 1 2 3 4 5
0 ~ ~ ~ ~ ~
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ ~ ~ ~
4 ~ ~ ~ ~ 0 0
5 ~ ~ ~ ~ 0 0

Enter: g row col (to guess), b row col (to bomb, 0 left), or q (to quit): q
Oh! I have to leave now. See you later.
cos135@NRC: /media/sf_COS135_VM_Codes/PS
cos135@NRC: /media/sf_COS135_VM_Codes/PS
cos135@NRC: /media/sf_COS135_VM_Codes/PS
cos135@NRC: /media/sf_COS135_VM_Codes/PS
```

Grading Rubric:

1. Board Setup (20 pts)
 - a. Board correctly implemented as 6×6 (5 pts)
 - b. Ships randomly placed (different each run) (5 pts)
 - c. Exactly 5 ships placed (2 pts)
 - d. No overlapping ships (3 pts)
 - e. Board initialized and displayed correctly (~, X, 0) (5 pts)
2. Game Play Logic (20 pts)
 - a. User can enter row and column correctly (5 pts)
 - b. Hit detection works correctly (X displayed) (5 pts)
 - c. Miss detection works correctly (0 displayed) (5 pts)
 - d. Prevents repeated guesses with warning (5 pts)
3. Bomb Feature (20 pts)
 - a. Bomb feature implemented and usable (5 pts)
 - b. Correct 3×3 area affected (5 pts)
 - c. Handles board boundaries correctly (no crashes) (5 pts)
 - d. Bomb count updates correctly and limited to 2 uses (5 pts)
4. Input Validation & Error Handling (5 pts)
 - a. Handles invalid inputs (non-numeric, out-of-range) (3 pts)
 - b. Program does not crash and prompts user again (2 pts)
5. Code Structure & Readability (10 pts)
 - a. Proper use of functions (3 pts)
 - b. Meaningful variable names (2 pts)
 - c. Code is well-indented and readable (2 pts)
 - d. Required comments included (3 pts)
6. Video Demonstration (5 pts)
 - a. Clearly explains code and logic (2 pts)
 - b. Demonstrates program running (2 pts)
 - c. Demonstrates understanding; does not simply read from code (1 pt)